Exercice pratique : automatiser lint + tests sur GitHub

Le but : jusqu'à maintenant vous lanciez `npm run lint` et les tests **à la main**. Vous allez créer un pipeline qui les lance **tout seul, sur GitHub, à chaque `push` et chaque pull request**. À la fin, votre dépôt affichera une CI **verte** quand tout va bien, **rouge** quand un test casse.

Prérequis : votre projet doit déjà avoir les scripts `lint` et `test` qui fonctionnent.

Étape 1 — Créer le fichier du workflow

Créez, à la racine du projet, un dossier `.github`, un sous-dossier `workflows`, et dedans un fichier `ci.yml`. Le chemin final doit être exactement : `.github/workflows/ci.yml`

> Attention, le dossier s'appelle bien `.github` **avec le point au début**. Le nom du fichier peut être ce que vous voulez, mais l'extension doit être `.yml`.

Étape 2 — Décrire le pipeline

Collez ce contenu dans `ci.yml` :

```
name: CI

on:
  push:
  pull_request:

jobs:
  verification:
    runs-on: ubuntu-latest
    steps:
      - name: Récupérer le code
        uses: actions/checkout@v4

      - name: Installer Node.js
        uses: actions/setup-node@v4
        with:
```

```
          node-version: 20

      - name: Installer les dépendances
        run: npm install

      - name: Lancer le linter
        run: npm run lint

      - name: Lancer les tests
        run: npx vitest run
```

> **Aide — lecture du fichier :** `on:` liste **quand** ça se déclenche (ici : à chaque push et chaque PR). `runs-on:` = la machine utilisée. `steps:` = la liste des actions, dans l'ordre : récupérer le code → installer Node → installer les dépendances → linter → tests.
>
> Attention → Le YAML est **sensible à l'indentation** : uniquement des espaces, jamais de tabulations.

Étape 3 — Envoyer le workflow sur GitHub

Sur une branche (pas `main` !), ajoutez le fichier, committez et poussez.

```
git checkout -b ajout-ci
git add .github/workflows/ci.yml
git commit -m "Ajout du pipeline CI"
git push -u origin ajout-ci
```

Puis ouvrez une **pull request** comme au TP1.

Étape 4 — Regarder le pipeline tourner

> Sur GitHub, allez dans l'onglet **Actions** du dépôt : vous voyez votre workflow en cours (rond jaune qui tourne), puis ✅ vert s'il réussit. Dans votre **pull request**, une section « checks » apparaît aussi en bas et indique si la CI passe.

Étape 5 — Casser un test et voir la CI rouge

Sur la même branche, cassez volontairement un test (comme au TP3 : modifiez le composant pour que le résultat soit faux), committez et poussez.

> Après le `git push`, retournez dans l'onglet **Actions** ou dans votre PR : le pipeline doit passer ❌ **rouge**. Cliquez sur le job en échec → dépliez l'étape « Lancer les tests » : vous voyez **le même message d'erreur** qu'en local, mais cette fois c'est GitHub qui l'a détecté automatiquement. Dans la PR, la fusion est signalée comme risquée.

Étape 6 — Réparer et revenir au vert

Corrigez le code, committez, poussez. La CI se relance seule et repasse au **vert**.

> **Si vous avez fini en avance :**
>
> **Bonus 1 —** Ajoutez une **règle de protection** sur `main` (Settings → Branches → Add rule) qui **exige que la CI passe** avant de pouvoir fusionner une PR. Testez : une PR rouge devient impossible à fusionner.
>
> **Bonus 2 —** Ajoutez le **badge de statut** de la CI (vert/rouge) en haut de votre `README.md` (cherchez « GitHub Actions status badge »).